UNIVERSIDAD INTERAMERICANA DE PUERTO RICO · RECINTO DE FAJARDO

PROYECTO FINAL · DESARROLLO WEB FULL-STACK

Sabías que PR

Documento final del proyecto

Aplicación web full-stack de noticias, educación y debate comunitario

Estudiante	William R. Collado
Número de estudiante	M00105850
Curso	COMP2053 — Desarrollo Web Full-Stack
Profesora	Milagros Donato Cintrón
Institución	Universidad Interamericana de Puerto Rico, Recinto de Fajardo
Sitio en línea	https://sabiasquepr.com
Repositorio	github.com/Wcollado72/SabiasquePR
Fecha	23 de octubre de 2026

1. Introducción

Sabías que PR es una plataforma web nacida en Puerto Rico que reúne noticias con contexto, contenido educativo y espacios de debate comunitario, con una misión sin fronteras: ayudar a cualquier persona a entender su entorno. Es, al mismo tiempo, un producto real en construcción —que se lanzará públicamente cuando tenga suficiente contenido— y el proyecto final del curso **COMP2053 Desarrollo Web Full-Stack**.

Este documento presenta el proyecto de principio a fin: los objetivos y la justificación, las tecnologías elegidas, la arquitectura, el diseño de la base de datos, capturas del sistema funcionando en línea, la evidencia de pruebas y las conclusiones. Acompañan a este documento el repositorio con las carpetas `/frontend` y `/backend`, el manual de usuario, la documentación técnica, el video demostrativo y la aplicación desplegada en `sabiasquepr.com`.

2. Objetivos

Objetivo general

Diseñar, desarrollar, probar y desplegar una aplicación web full-stack funcional que integre un frontend responsivo, una API REST propia con operaciones CRUD completas, una base de datos relacional persistente y un sistema de autenticación con JWT verificado en el servidor.

Objetivos específicos

- Construir una API REST organizada por rutas, con validación centralizada de datos, manejo de errores y rutas protegidas por rol.
- Diseñar una base de datos PostgreSQL normalizada de 15 tablas, con llaves primarias y foráneas, restricciones y seguridad por fila.
- Implementar registro, inicio de sesión y verificación en dos pasos, validando cada JWT en el servidor.
- Desarrollar un frontend responsivo, accesible y bilingüe que consuma la API y ofrezca una buena experiencia de usuario.
- Verificar el sistema con pruebas automáticas, integración continua y pruebas en distintos navegadores y dispositivos, y publicarlo en línea.

3. Justificación

Un proyecto final full-stack evalúa que se construyan y se controlen las tres capas de una aplicación real: un backend propio, una base de datos con esquema propio y un frontend que consuma esa API, no solo enlaces a servicios externos. El sitio responde además a una necesidad concreta: un espacio en español, desde Puerto Rico, donde las noticias vienen con contexto y la comunidad puede debatir con respeto y moderación.

El debate, la gestión de contenidos y la tienda sirven de eje del CRUD, el criterio de mayor peso de la rúbrica (30 %). Las decisiones de diseño —todo el contenido es gratis, el apoyo es voluntario, las secciones para adultos tienen verificación de edad y la protección de menores es prioridad— reflejan que el proyecto está pensado para operar como un servicio real y no solo para aprobar un curso.

4. Tecnologías utilizadas

Capa	Tecnología	Uso en el proyecto
Frontend	HTML5, CSS3, JavaScript (sin framework)	Sitio responsivo generado por un script de plantillas (build.js), PWA con service worker, modo claro/oscuro, bilingüe ES/EN
Backend	Node.js 22 + Express 5	API REST propia con 54 endpoints, validadores, middleware de autenticación y errores centralizados
Base de datos	PostgreSQL (Supabase)	15 tablas relacionales con Row Level Security activo
Autenticación	Supabase Auth + jose (JWKS, ES256)	Registro, inicio de sesión, Google y 2FA; el servidor verifica cada JWT
Pagos	Stripe	Tienda y apoyo voluntario con webhook firmado (modo de prueba)
Servicios	Resend, Cloudflare Turnstile, Beehiiv, Formspree, API de Claude	Correos de autenticación, anti-robots, boletín, contacto y agentes de IA del panel
Calidad	Jest + Supertest, ESLint	774 pruebas automáticas y revisión de estilo en cada cambio
Control de versiones y CI	Git, GitHub, GitHub Actions	Ramas y pull requests; flujos de pruebas, auditoría de dependencias y secretos, respaldo diario cifrado
Despliegue	Netlify + Render + Supabase	Frontend estático con dominio propio, API y base de datos en servicios gestionados

5. Arquitectura del sistema

El sistema sigue una arquitectura de tres capas. El navegador carga el frontend estático y se autentica directamente con Supabase Auth, que le entrega un JWT firmado. El frontend consume la API REST propia enviando ese JWT en el encabezado `Authorization: Bearer`; el backend lo valida contra las llaves públicas del proyecto (JWKS) y solo entonces consulta PostgreSQL con una conexión directa. La API de datos de Supabase permanece cerrada.

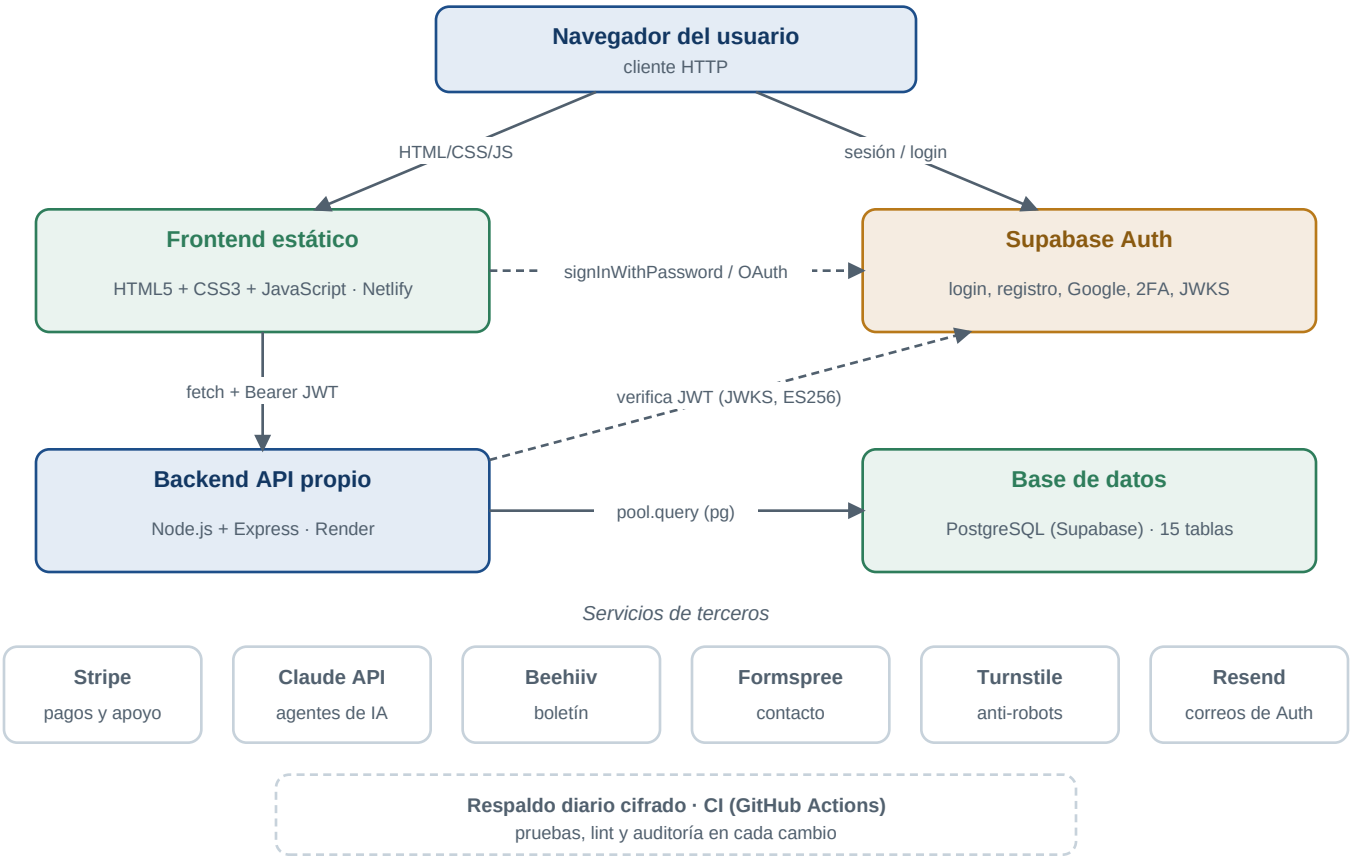


Figura 1. Arquitectura de tres capas con Supabase Auth como servicio de identidad compartido.

Navegación del sitio

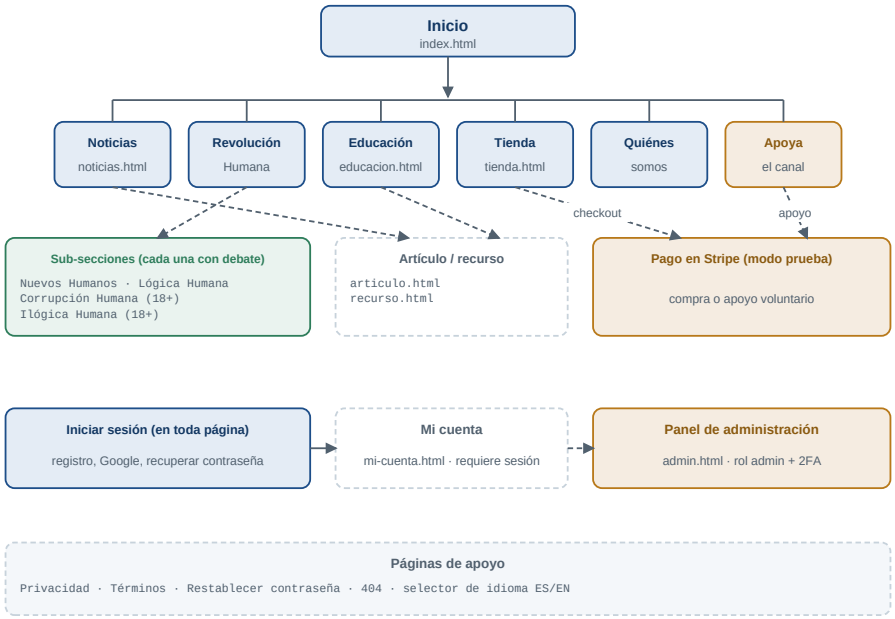


Figura 2. Mapa de navegación: páginas públicas, área con sesión y área restringida por rol.

Funcionalidad CRUD y rutas protegidas

La API expone **54 endpoints**. Cada recurso implementa las operaciones que le corresponden: noticias, contenidos, productos y publicaciones del debate tienen CRUD completo. Las rutas de escritura exigen un JWT válido (401 si falta o es inválido) y el rol necesario (403 si no lo tiene). La siguiente matriz se genera a partir del código de las rutas.

Recurso (módulo)	Crear	Consultar	Modificar	Eliminar	Acceso
admin	POST	GET	PUT	DELETE	Admin, Moderador/Admin
agradecimientos	—	GET	—	—	Público
comentarios	POST	GET	—	DELETE	Público, Sesión
contenidos	POST	GET	PUT	DELETE	Admin, Opcional
foro	POST	GET	PATCH	DELETE	Público, Sesión
newsletter	POST	—	—	—	Público
noticias	POST	GET	PUT	DELETE	Admin, Opcional
ordenes	—	GET	—	—	Sesión
perfil	POST	GET	PUT	DELETE	Sesión
productos	POST	GET	PUT	DELETE	Admin, Público
reportes	POST	GET	PATCH	—	Moderador/Admin, Sesión
stripe	POST	—	—	—	Sesión
stripeWebhook	POST	—	—	—	Firma de Stripe (sin sesión)

Acceso: Público = sin sesión; Opcional = con o sin sesión; Sesión = JWT válido; Moderador/Admin y Admin = además exigen ese rol.

6. Diseño de la base de datos

La base de datos tiene **15 tablas**, 15 llaves primarias, **11 llaves foráneas** y 31 restricciones CHECK, normalizadas a Tercera Forma Normal, con Row Level Security activo. `perfiles` extiende la identidad de Supabase Auth y es el origen de las relaciones de autoría. El diccionario de datos completo, el esquema relacional y el script de creación están en el documento de Planificación y Diseño de la Base de Datos que acompaña la entrega.

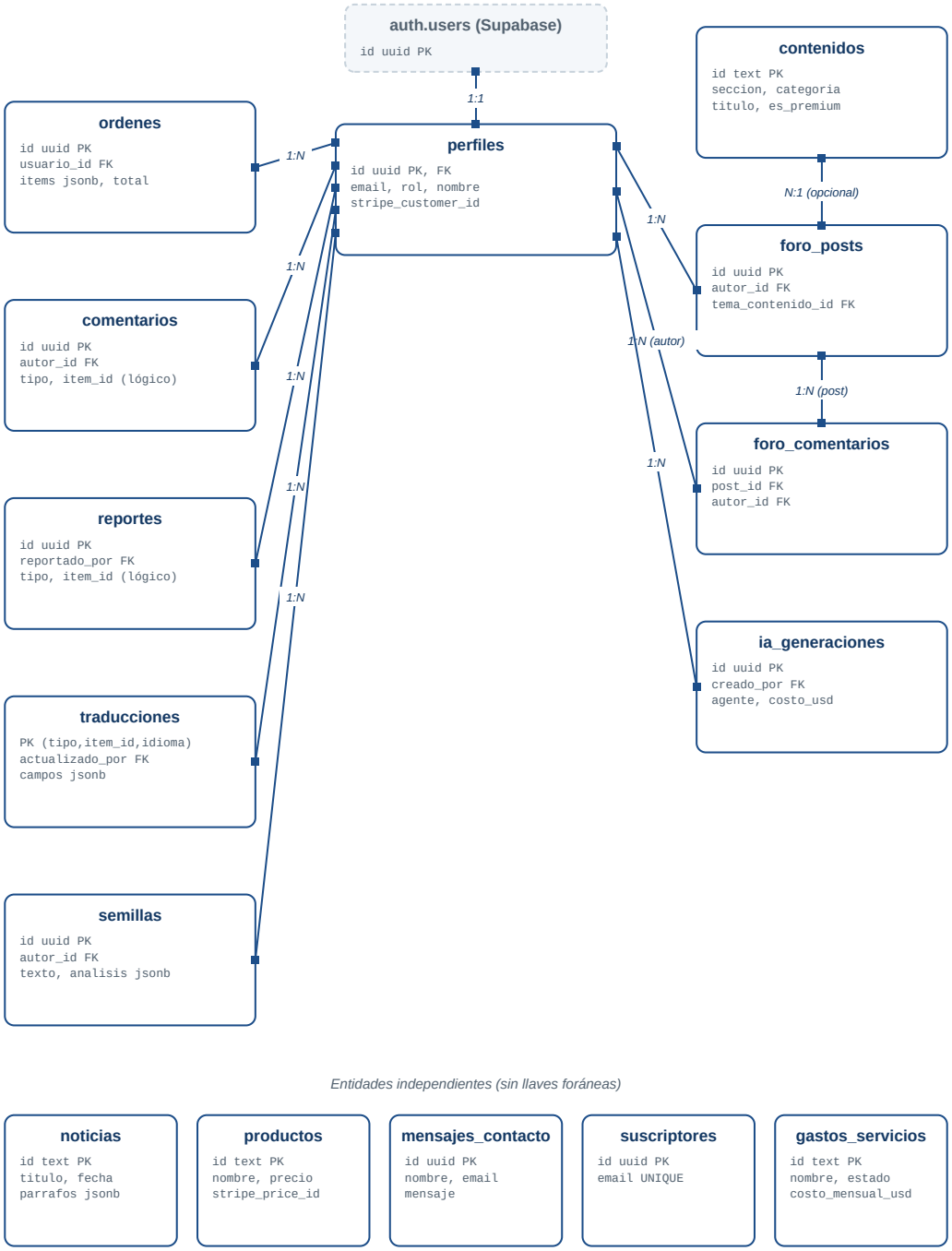


Figura 3. Modelo Entidad-Relación de las 15 tablas propias.

Tabla	Propósito
comentarios	Comentarios bajo una noticia o un contenido. La pieza comentada se identifica por (tipo, item_id), sin llave foránea.

Tabla	Propósito
contenidos	Contenido multi-formato (video, ensayo, noticia, lectura corta, audio, foto) que alimenta Noticias, Revolución Humana y Educación.
foro_comentarios	Comentarios dentro de una publicación del debate.
foro_posts	Publicaciones del debate comunitario, asociadas a una sub-sección y, opcionalmente, a un contenido.
gastos_servicios	Inventario de los servicios de terceros del proyecto (estado del plan, costo mensual y renovación) para el panel de administración.
ia_generaciones	Registro de cada uso de los agentes de IA de la Brigada Coquí (cada uno con su nombre): tokens y costo estimado.
mensajes_contacto	Mensajes recibidos desde el formulario de contacto.
noticias	Noticias rápidas del sitio (feed de Noticias y RSS). Cada fila es un artículo corto con su fuente.
ordenes	Compras de la tienda y apoyos procesados por Stripe, con el estado del pago y las direcciones del cliente.
perfiles	Extiende la identidad de Supabase Auth (auth.users) con el rol, los datos de la cuenta y la información de Stripe. Es el origen de todas las relaciones de autoría.
productos	Artículos de la tienda. Se leen públicamente; solo el administrador los crea, modifica o elimina.
reportes	Denuncias de publicaciones o comentarios (del debate y de noticias/contenidos), incluidas las que genera el filtro automático, para la moderación.
semillas	Cuaderno de ideas propias del autor (Aliado Imperfekto): cada semilla guarda el texto, sus etiquetas, el estado y los análisis de las agentes del Taller.
suscriptores	Correos suscritos al boletín.
traducciones	Versión en inglés de noticias, contenidos y productos. Llave compuesta (tipo, item_id, idioma).

7. Capturas del sistema en funcionamiento

Capturas tomadas en la aplicación desplegada (`https://sabiasquepr.com`). Cada una muestra una pantalla real servida desde producción.

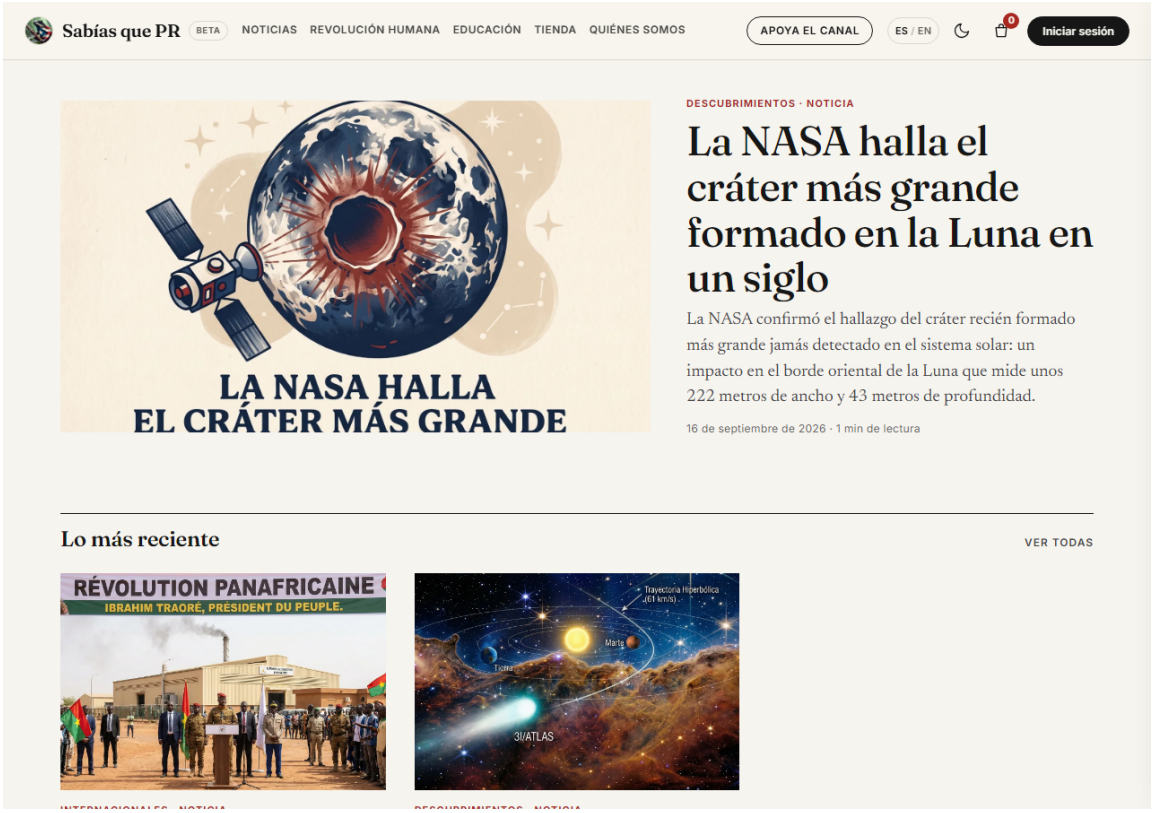


Figura 4. Página de inicio.

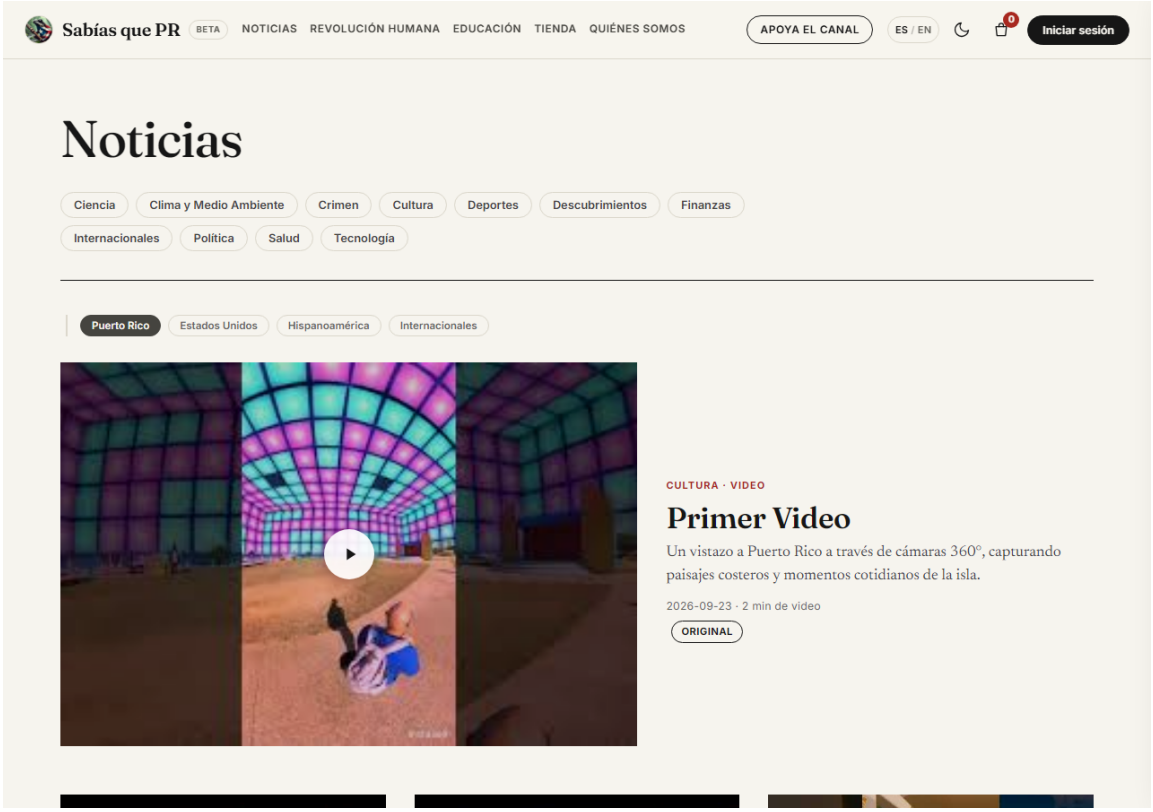


Figura 5. Noticias, con filtros por categoría y por región.

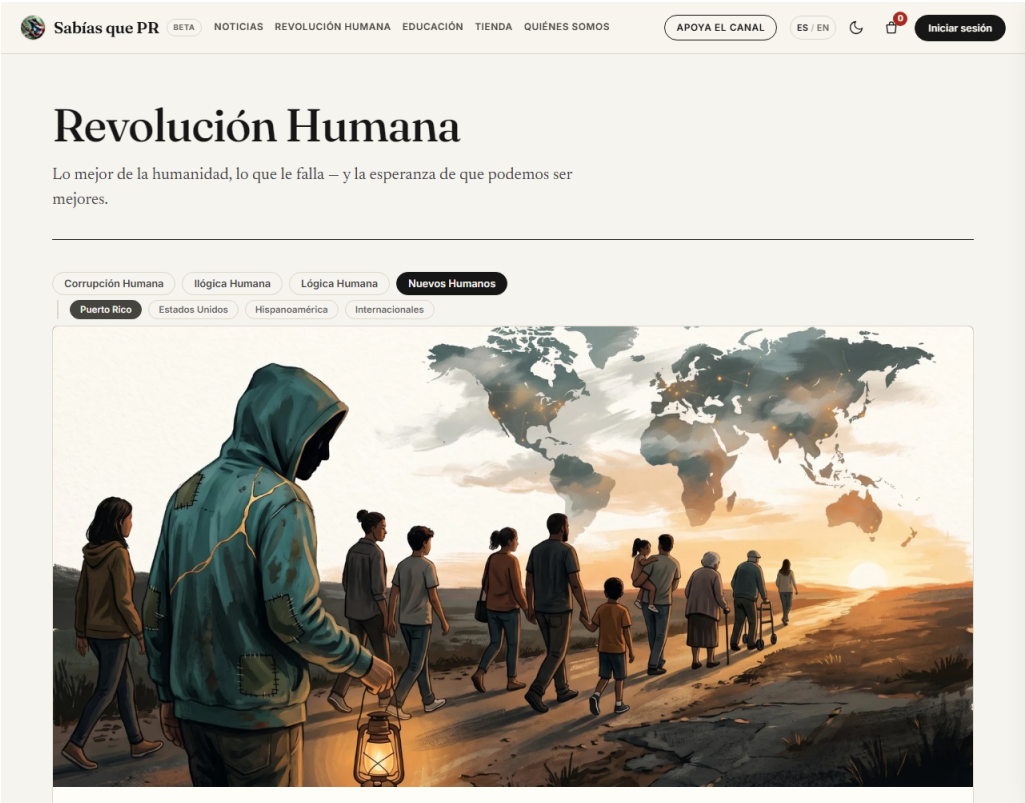


Figura 6. Revolución Humana y sus sub-secciones.

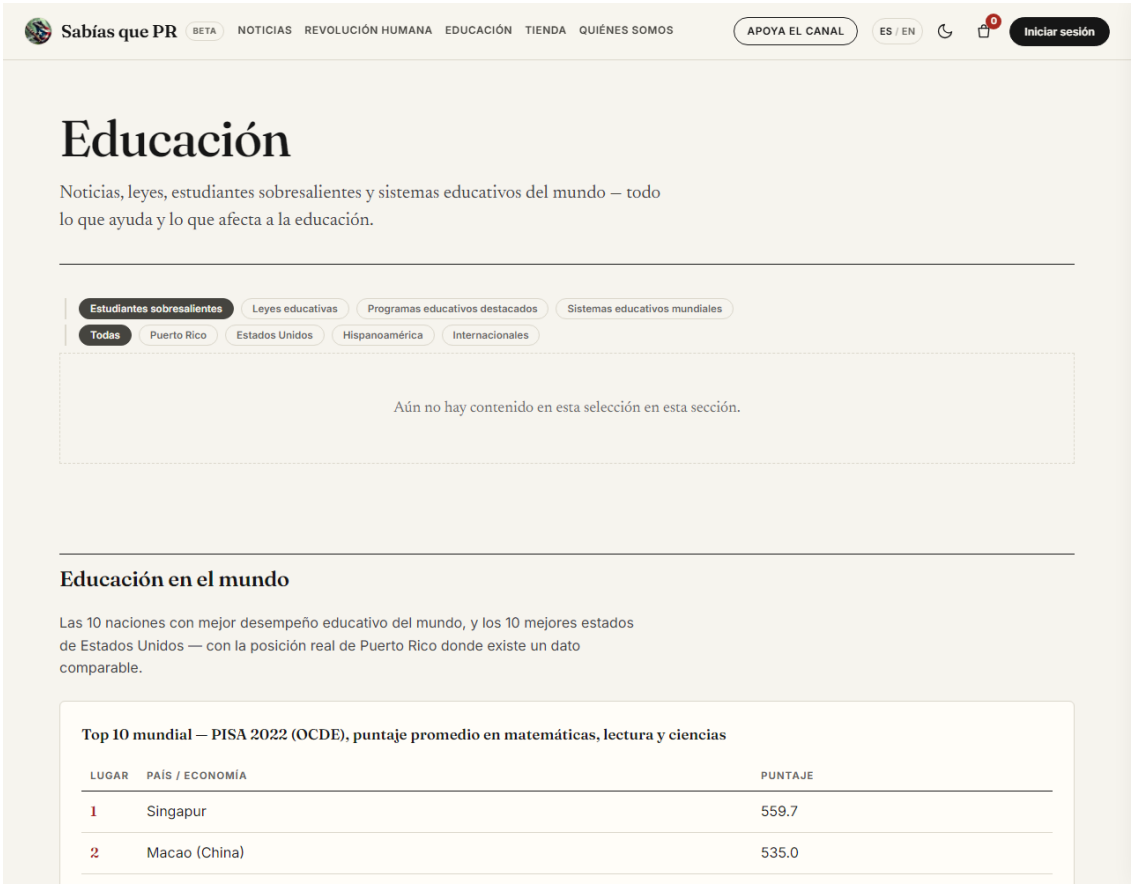


Figura 7. Educación, con filtros de tema y de región.

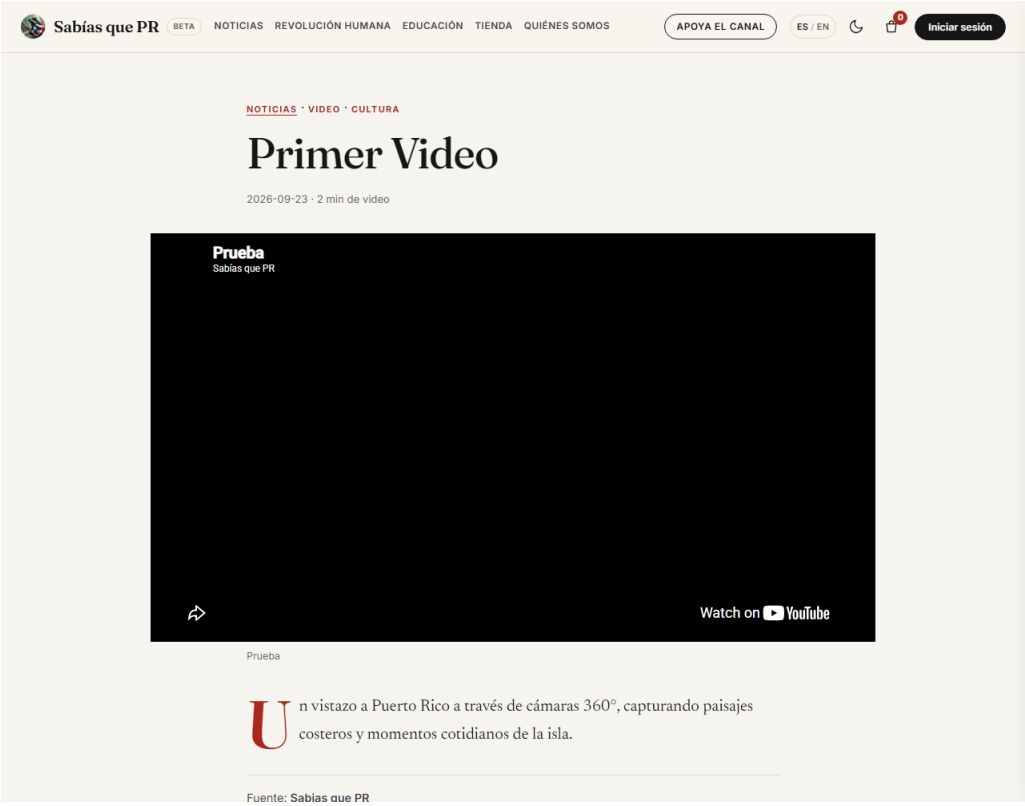


Figura 8. Detalle de una pieza (video).

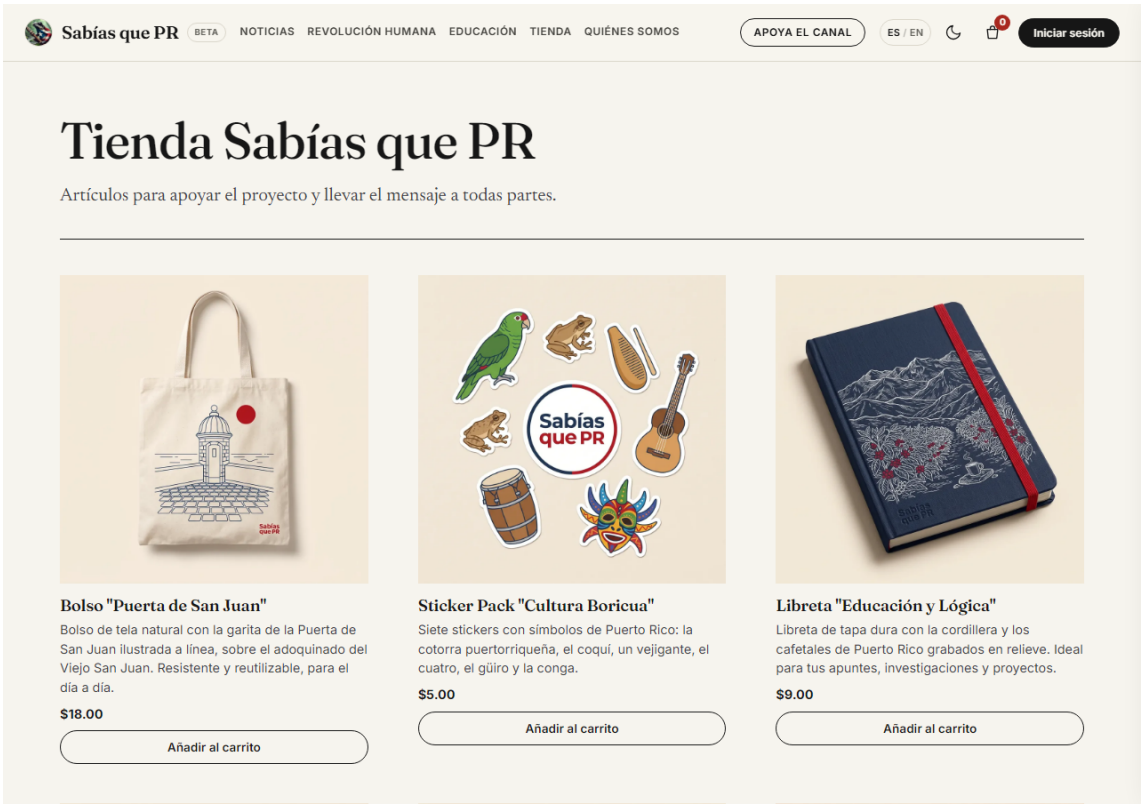


Figura 9. Tienda.



Figura 10. Apoya el canal (apoyo voluntario).



Figura 11. Quiénes somos y contacto.

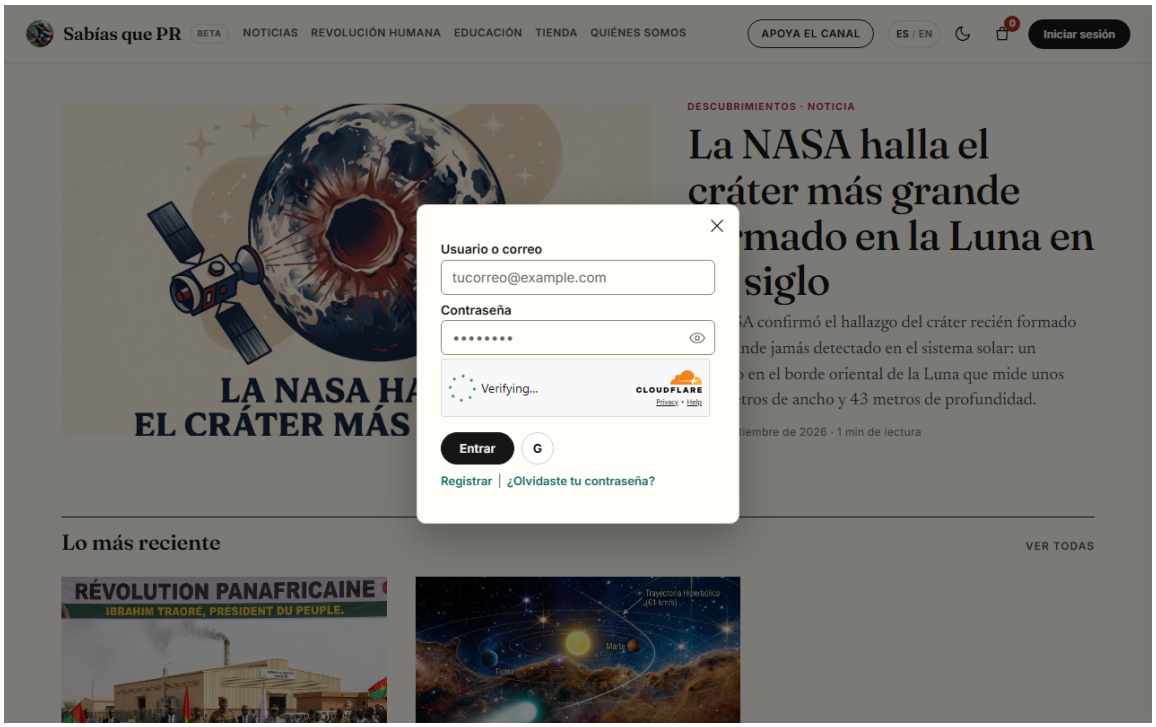


Figura 12. Inicio de sesión con verificación anti-robots.

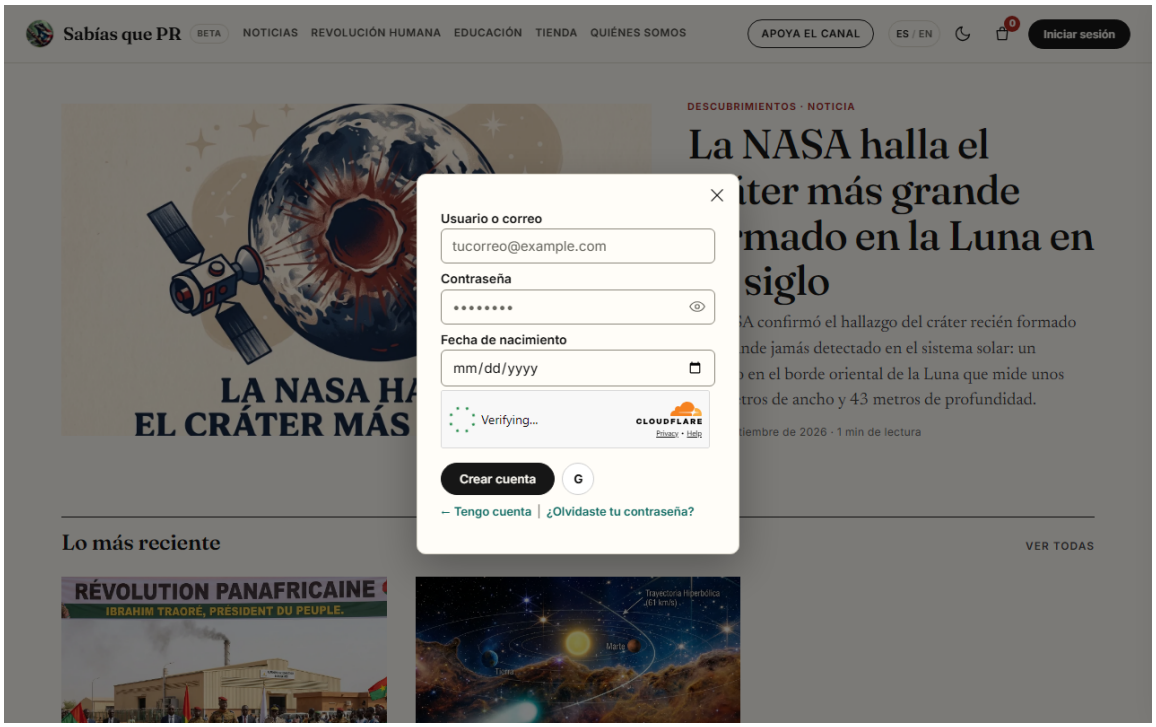
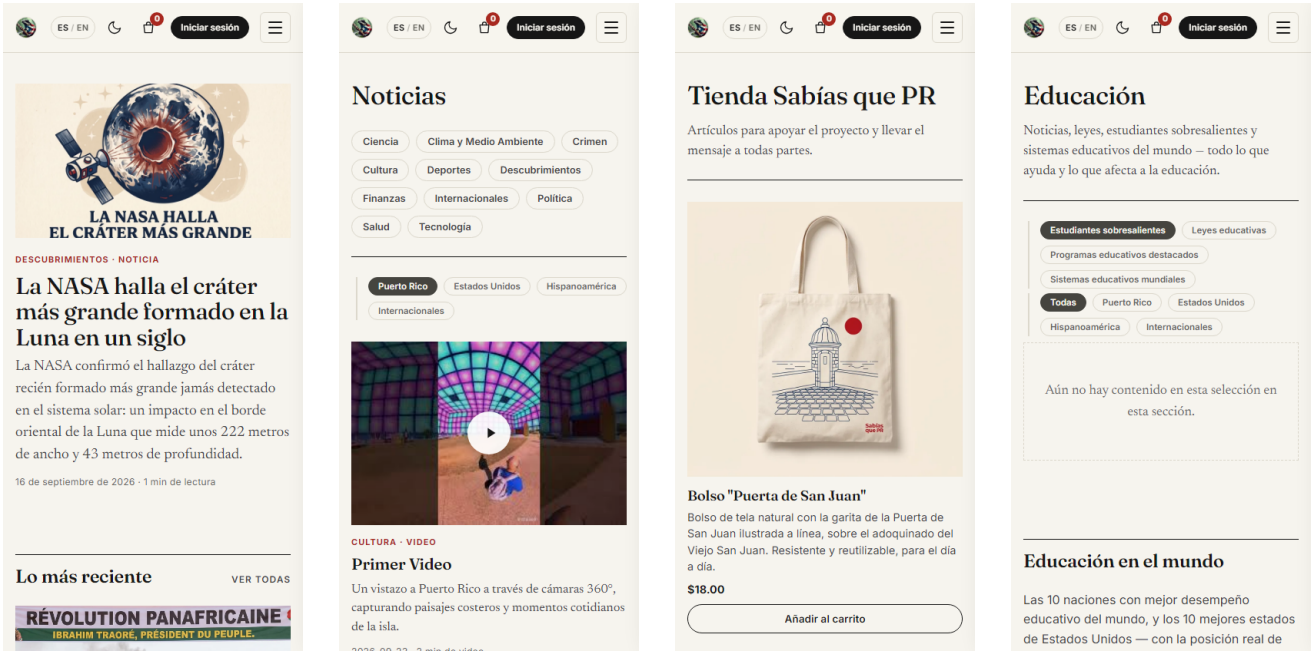


Figura 13. Creación de cuenta.

Vistas en teléfono móvil (diseño responsivo)



Inicio *Noticias* *Tienda* *Educación*

Figura 14. El mismo sitio en una pantalla de 390 px de ancho.

8. Evidencias de pruebas

Las pruebas automáticas (Jest y Supertest) suman **774 pruebas en 57 archivos**; **774 pasan y 0 fallan**. La cobertura global es de **87.24 % de las sentencias** (82.65 % de las ramas, 87.36 % de las funciones y 88.53 % de las líneas). Se ejecutan en cada cambio mediante GitHub Actions junto con ESLint, la auditoría de dependencias y el análisis de secretos.

Cubren la autenticación (JWT inválido o ausente → 401; rol insuficiente → 403), el CRUD de cada recurso, la validación de datos, el webhook de Stripe con firma, las cabeceras de seguridad, la configuración de despliegue y el esquema de la base de datos. El detalle por archivo, la verificación contra producción y las pruebas por navegador y dispositivo están en el documento *Evidencia de pruebas* que acompaña la entrega.

Durante el proyecto se corrigieron errores reales encontrados al probar (por ejemplo, una diferencia entre las variables de entorno de producción y las de pruebas, un bloqueo de construcción en Netlify y textos sin traducir); cada uno quedó registrado en el historial de Git y, cuando aplicaba, cubierto con una prueba nueva para que no vuelva a ocurrir.

9. Conclusiones

El proyecto cumple los requisitos técnicos de la rúbrica: un frontend responsivo que consume una API REST propia de 54 endpoints, una base de datos relacional de 15 tablas, autenticación con JWT verificado en el servidor, rutas protegidas, CRUD completo y una aplicación desplegada y accesible en línea. Además de lo exigido, incorpora verificación en dos pasos, moderación del debate, pagos de prueba con Stripe, bilingüismo, transparencia sobre el uso de IA (con los iconos de las herramientas usadas), integración continua y respaldos cifrados.

Aprendizajes

- Separar responsabilidades en tres capas permite probar cada una por separado y cambiar un servicio sin reescribir el resto.
- La seguridad no puede quedarse en el navegador: validar el JWT en el servidor y cerrar la API de datos fue una decisión central.
- Los servicios gratuitos tienen límites (pausa por inactividad, arranque lento): fue necesario documentarlos, automatizar un chequeo diario y practicar una migración completa a un proyecto nuevo de Supabase con plan de vuelta atrás.
- Las pruebas automáticas y la integración continua evitaron que los cambios rompieran lo que ya funcionaba, incluida la reestructuración final del repositorio en /frontend y /backend.

Trabajo futuro

- Activar Stripe en modo real y completar la entidad legal que respalda el proyecto antes del lanzamiento público.
- Unificar las dos tablas de noticias en un solo modelo de contenido.
- Debates en vivo tipo chat y más zonas de debate; que los agentes de IA pregunten idioma, formato y tono antes de investigar.

- Lanzamiento público cuando cada sección tenga suficiente contenido.

10. Referencias

Anthropic. (s. f.). *Claude API documentation*. https://docs.anthropic.com/

Cloudflare. (s. f.). *Turnstile documentation*. https://developers.cloudflare.com/turnstile/

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM, 13*(6), 377–387. https://doi.org/10.1145/362384.362685

Express. (s. f.). *Express: Fast, unopinionated, minimalist web framework for Node.js*. https://expressjs.com/

Federal Trade Commission. (s. f.). *Children's Online Privacy Protection Rule (COPPA)*. https://www.ftc.gov/legal-library/browse/rules/childrens-online-privacy-protection-rule-coppa

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Jest. (s. f.). *Jest documentation*. https://jestjs.io/docs/getting-started

Jones, M., Bradley, J., y Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc7519

Mozilla. (s. f.). *MDN Web Docs*. https://developer.mozilla.org/

Netlify. (s. f.). *Netlify documentation*. https://docs.netlify.com/

Node.js. (s. f.). *Node.js documentation*. https://nodejs.org/docs/latest/api/

OWASP Foundation. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/

PostgreSQL Global Development Group. (s. f.). *PostgreSQL documentation*. https://www.postgresql.org/docs/

Render. (s. f.). *Render documentation*. https://render.com/docs

Stripe. (s. f.). *Stripe documentation*. https://docs.stripe.com/

Supabase. (s. f.). *Supabase documentation*. https://supabase.com/docs

World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/